

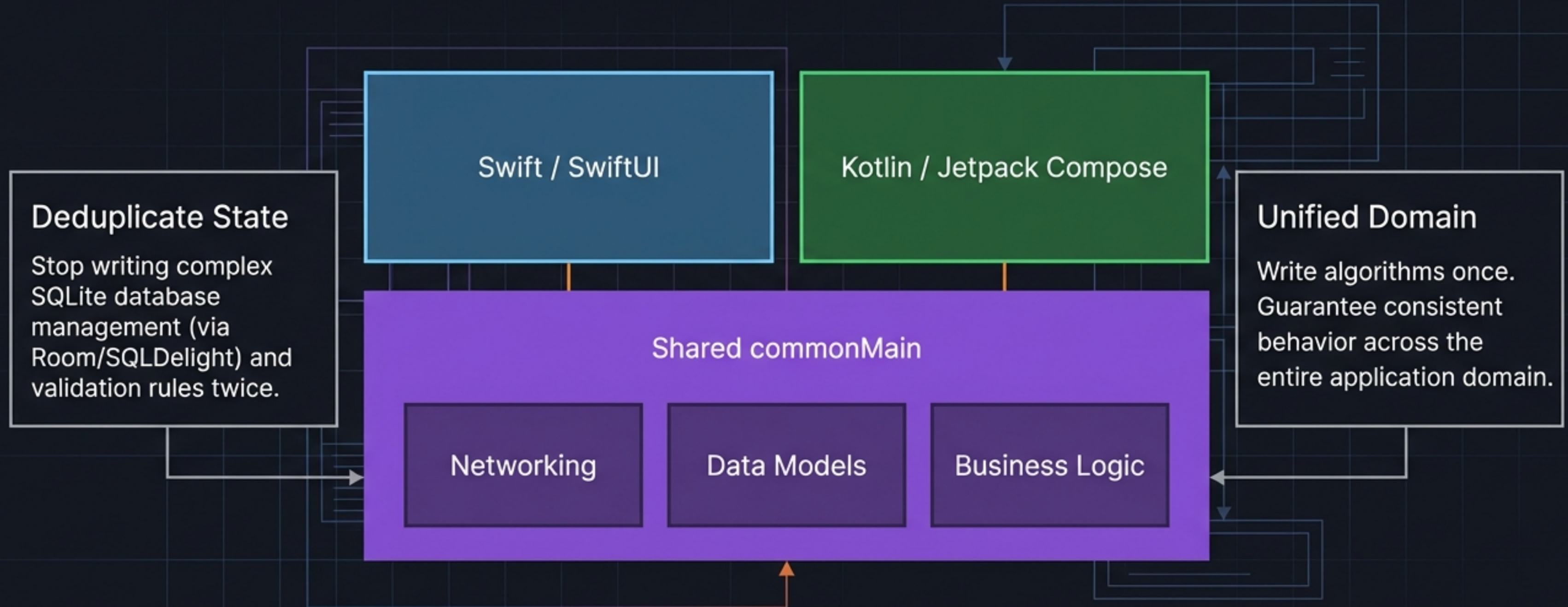
Kotlin Multiplatform (KMP): Native Performance, Shared Logic.

A technical overview of bridge-less code sharing for engineering teams.

Redefining cross-platform: Share what makes sense. Keep native what matters.

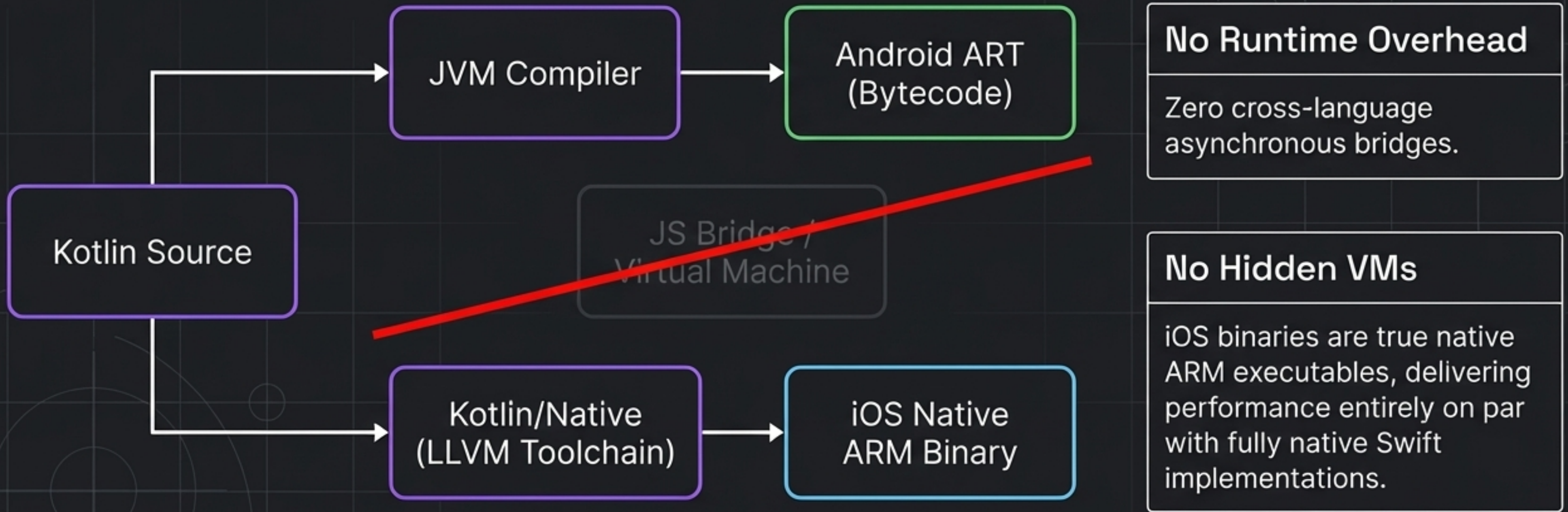
The Core Paradigm: Headless Logic Sharing

KMP is engineered to share business logic, data handling, and network requests across Android, iOS, web, and desktop.



Under the Hood: The Native Compilation Pipeline

KMP compiles into the native way the target platform runs code.



Direct Interop: No Wrappers Required

Bidirectional interoperability. Kotlin speaks directly to native APIs.

```
import platform.AVFoundation.AVCaptureSession
import platform.AVFoundation.AVMediaTypeVideo

fun startCameraSession() {
    val session = AVCaptureSession()
    session.sessionPreset = AVCaptureSessionPresetHigh
    session.startRunning()
}
```

Native iOS
AVFoundation API

Zero-Day Support

Call native APIs directly from Kotlin code without waiting for community-maintained wrapper plugins.

Seamless Translation

Make Kotlin functions available to Swift, and utilize the full power of iOS SDKs within Kotlin.

Architectural Safety: The expect/actual Escape Hatch

When bare-metal platform specifics are required, the framework elegantly steps out of your way.

commonMain

```
expect fun getDeviceName(): String
```

androidMain

```
actual fun getDeviceName() = Build.MODEL
```

iosMain

```
actual fun getDeviceName() =  
    UIDevice.currentDevice.name
```

Compiler-Enforced Safety

The Kotlin compiler mandates that every expect declaration in shared code has a corresponding actual implementation in platform modules.

Never Get Stuck

Retain the cross-platform approach without sacrificing low-level performance-critical native features.

Architectural Decision: The UI Flexibility Matrix

KMP dictates your data layer. Your presentation layer remains an engineering choice.

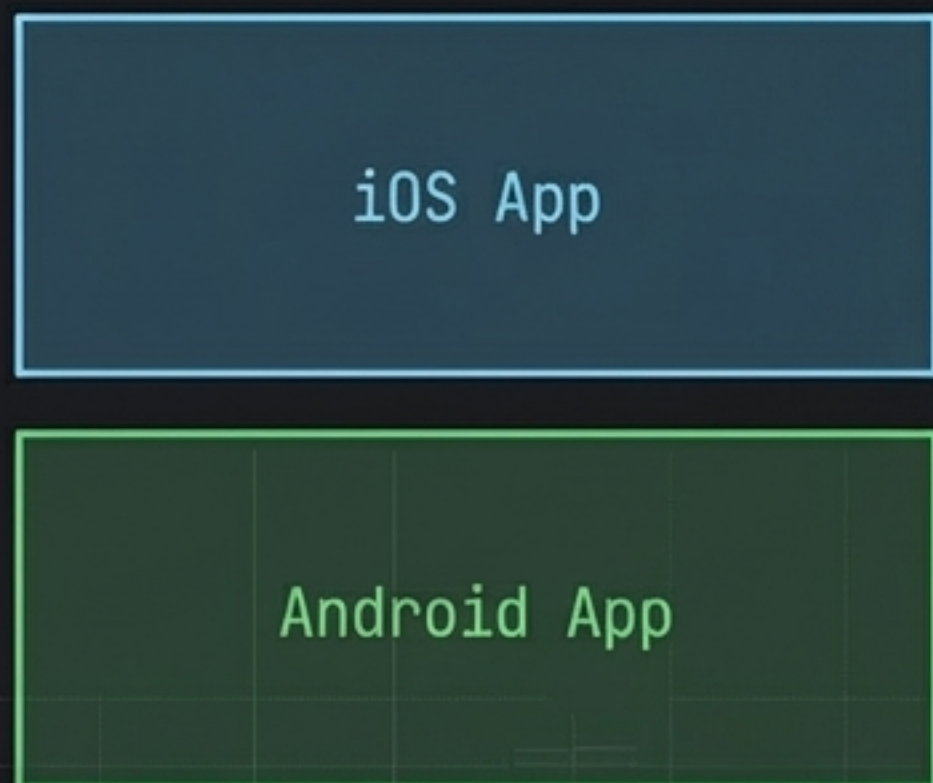
	Approach A: Shared Logic + Native UI (SwiftUI & Jetpack Compose)	Approach B: Compose Multiplatform (CMP) (Shared Logic + Shared UI)
UI Fidelity	100% Platform Fidelity	Native-feeling (Declarative UI framework)
Code Reuse	High (Business logic, data, networking)	Maximum (Up to 100% logic and UI)
Platform Knowledge Req	High (Requires native iOS/Android expertise)	Low (Kotlin-first)
Ideal Use Case	Strict UX precision, complex platform-specific animations.	Rapid feature delivery, internal tools, consistent cross-platform design systems.

KMP dictates your data layer. Your presentation layer remains an engineering choice.

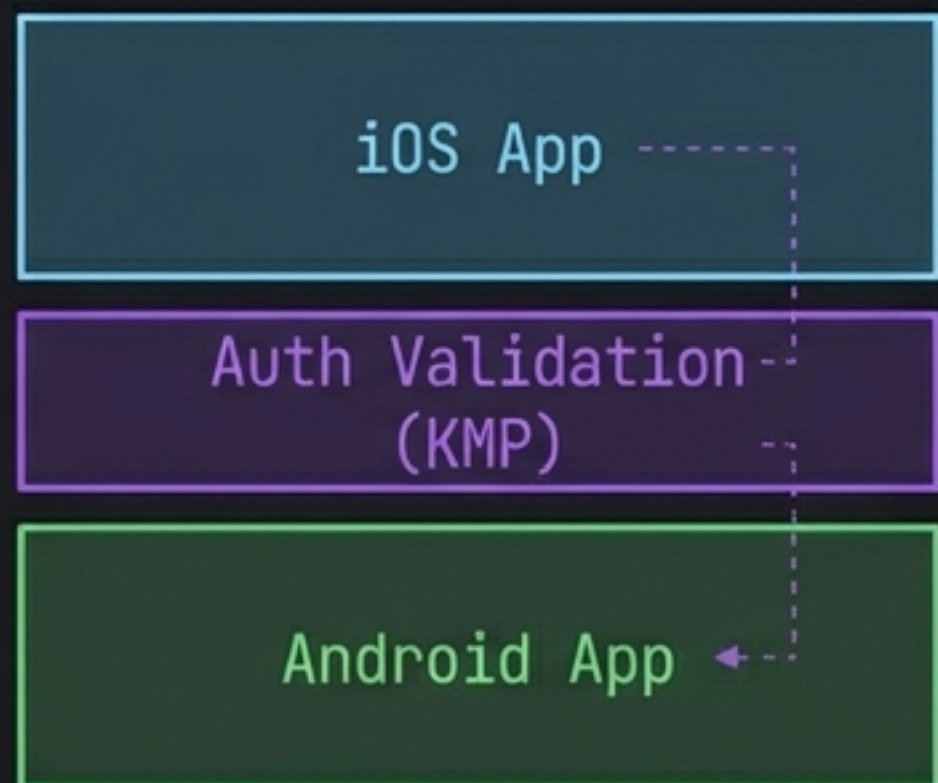
Risk Mitigation: Incremental Adoption

Gradual integration. Zero requirement to rewrite your existing app.

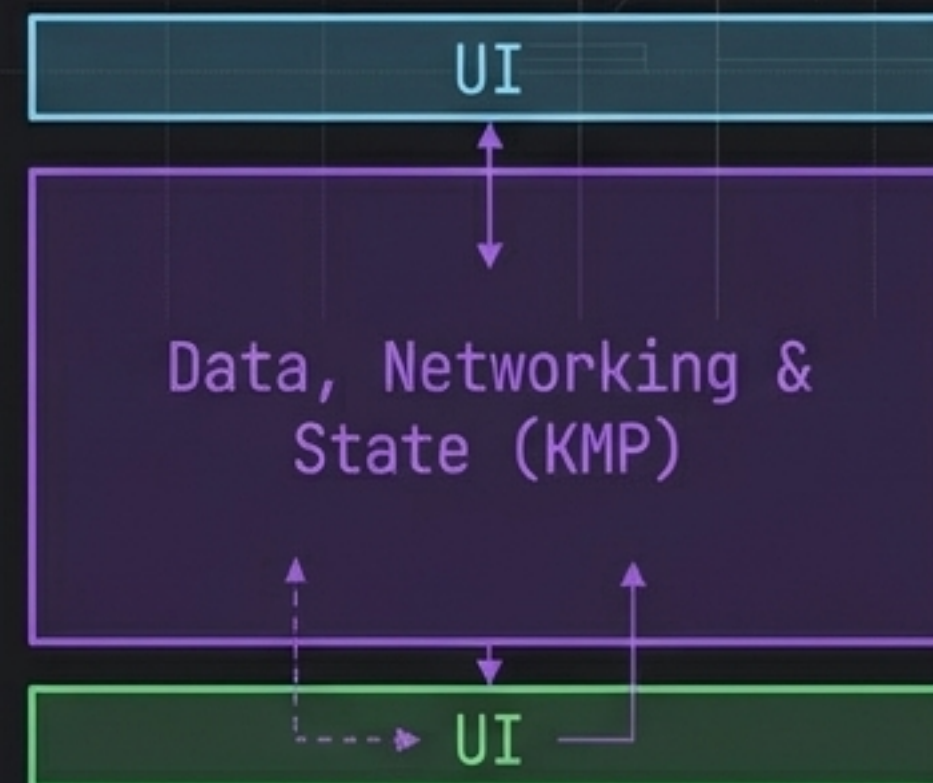
Step 1: 100% Native Codebases



Step 2: Targeted Injection



Step 3: Unified Data Layer



Module-by-Module

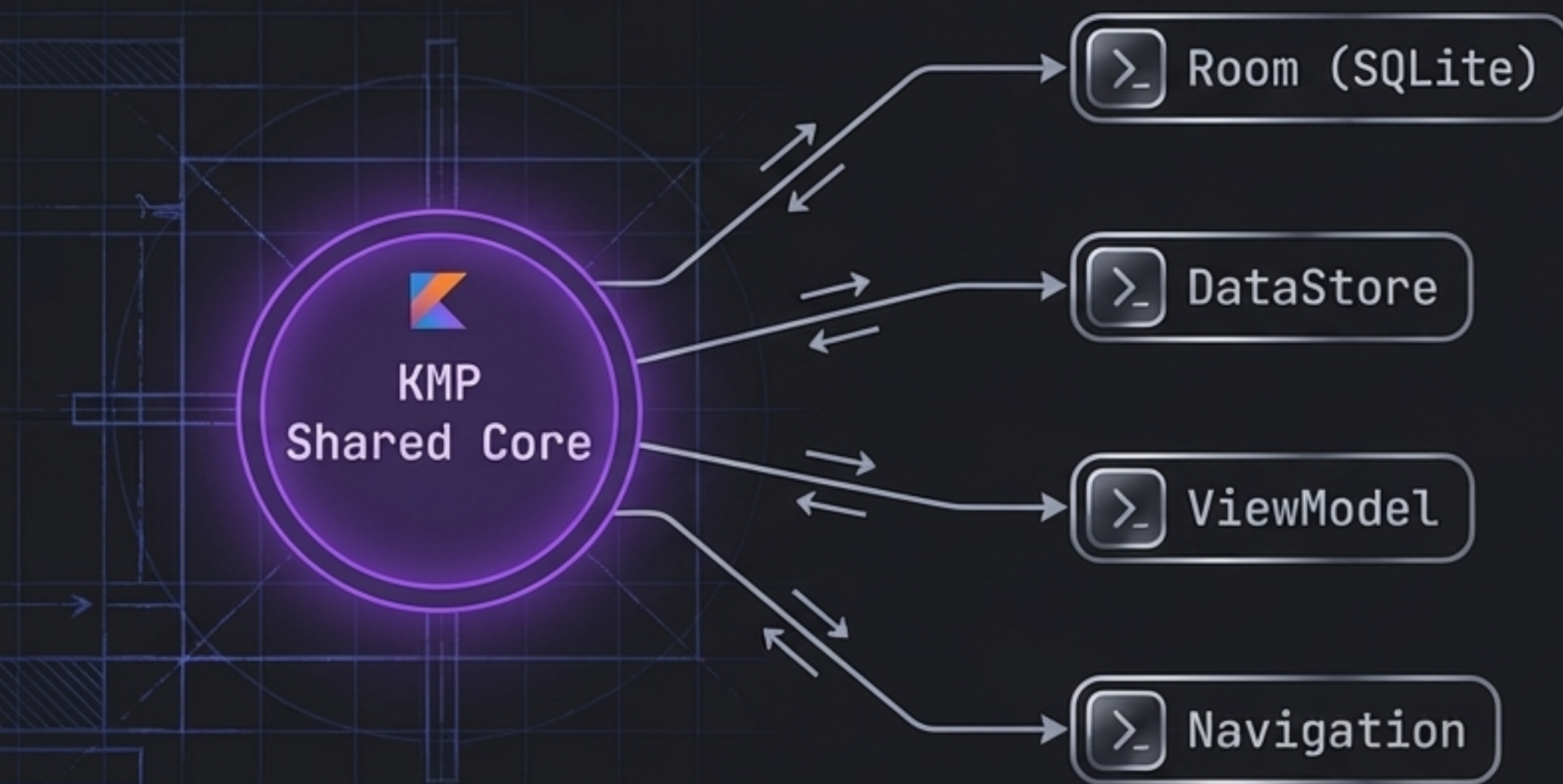
Start by isolating a single utility module (e.g., via the KMP module wizard in Android Studio).

Low-Risk Validation

Prove the tooling, CI/CD integration, and performance before committing to a full architectural shift.

Synthesis: The Production-Ready Stack

Kotlin Multiplatform is stable, supported, and treating iOS and Android as Tier-1 citizens.



Forbes

>80% Logic Shared

Enabled simultaneous feature rollouts across iOS and Android.

McDonald's

Unified Mobile Team

Resulted in fewer crashes and superior cross-platform performance.

Google Docs

Production Validated

Successfully implemented in high-stakes, real-world environments.

Go cross-platform without compromising engineering standards.